Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(Московский Политех)

Лабораторная работа № 2
По курсу: Технологии конструирования программного обеспечения

Выполнил: Стальмахов Иван Сергеевич
Группа: 241-3211
Проверил: _____

МОСКВА
2026

# Задание на лабораторную работу

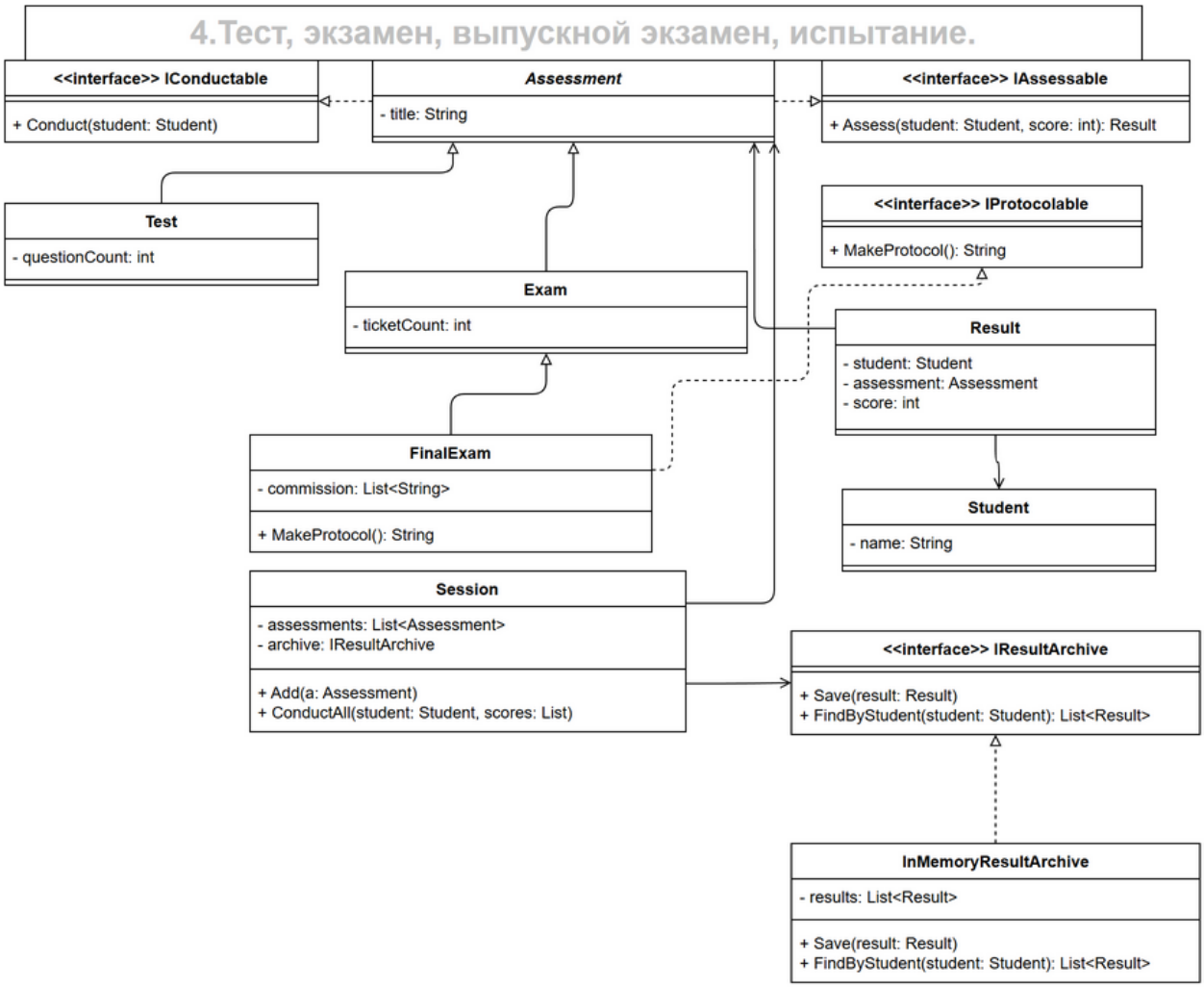
Разработка программ с использованием принципа подстановки Лисков (LSP), принципа разделения интерфейсов (ISP) и принципа инверсии зависимостей (DIP).

Вариант 4. Тест, экзамен, выпускной экзамен, испытание.

Требуется: UML-диаграмма классов и диаграмма последовательности; консольное приложение (C#) с применением LSP, ISP и DIP; тестирование функциональности созданных классов.

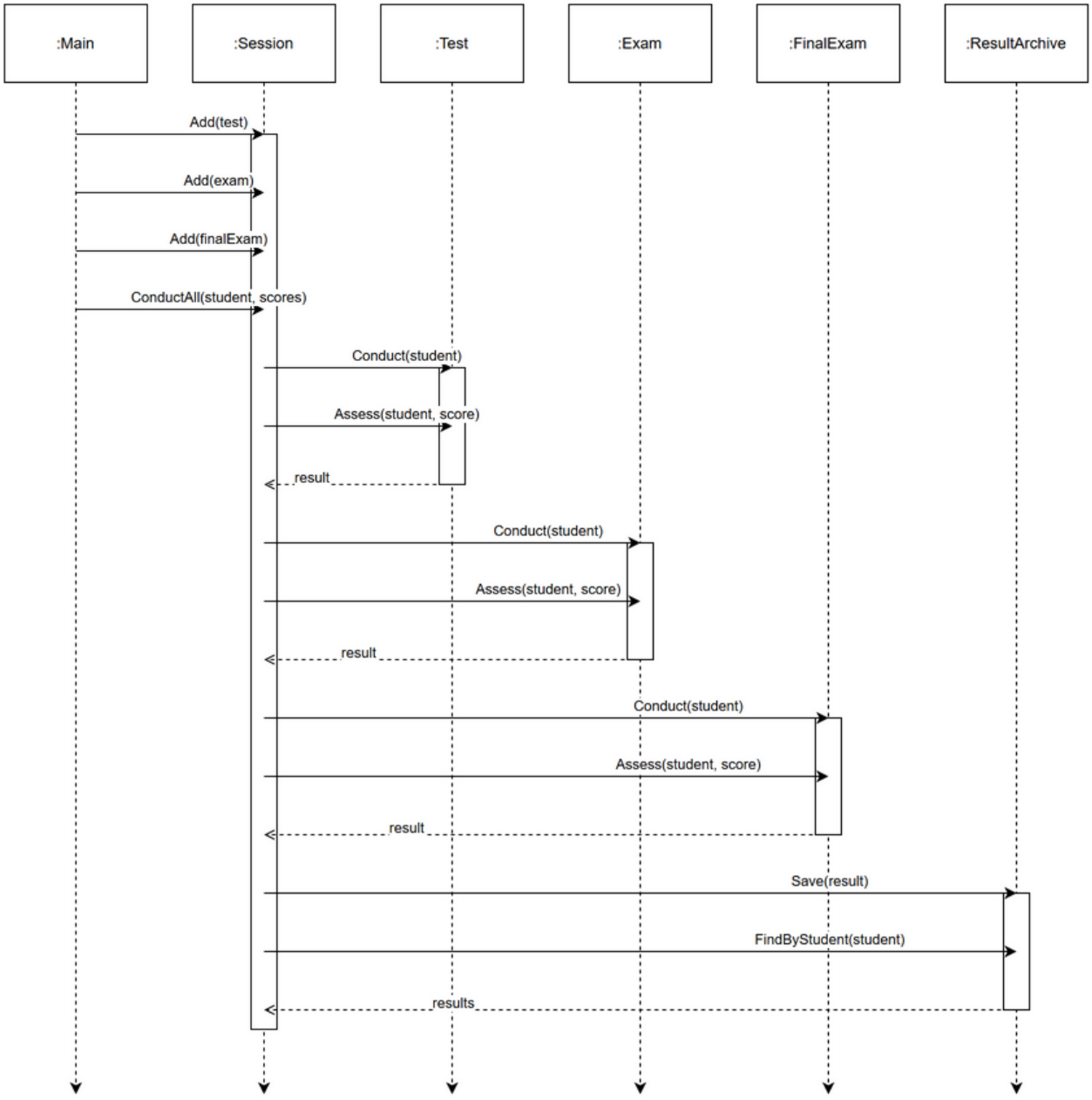
Реализация принципов в программе: LSP — `Test`, `Exam` и `FinalExam` взаимозаменяемы везде, где ожидается `Assessment` (`Session.ConductAll` работает со списком базового типа без проверок конкретного класса, наследники не ужесточают контракт базы); ISP — поведение разрезано на узкие интерфейсы `IConductable`, `IAssessable` и `IProtocolable`, протокол умеет только выпускной экзамен, остальные классы от него не зависят; DIP — `Session` хранит только абстракции (`List<Assessment>`, `IResultArchive`), конкретные классы создаются только в `Main`.

# Диаграмма классов



# Диаграмма последовательности

4.Тест, экзамен, выпускной экзамен, испытание.




# Исходный код программы

```
namespace Lab2;
public class Student(string name)
{
    public string Name { get; } = name;
```

```
    public override string ToString() => Name;
}
```

```
namespace Lab2;
public abstract class Assessment(string title) : IConductable,
   IAssessable
{
    public string Title { get; } = title;
    public abstract void Conduct(Student student);
    public abstract Result Assess(Student student, int score);
    public override string ToString() => Title;
}
```

```
namespace Lab2;
public class Test(string title, int questionCount) : Assessment(title)
{
    public int QuestionCount { get; } = questionCount;
    public override void Conduct(Student student)
    {
        if (QuestionCount <= 0) throw new InvalidOperationException("
   Test has no questions.");
    }
    public override Result Assess(Student student, int score) => new(
   student, this, score);
}
```

```
namespace Lab2;
public class Exam(string title, int ticketCount) : Assessment(title)
{
    public int TicketCount { get; } = ticketCount;
    public override void Conduct(Student student)
    {
        if (TicketCount <= 0) throw new InvalidOperationException("Exam
    has no tickets.");
    }
    public override Result Assess(Student student, int score) => new(
   student, this, score);
}
```

```
namespace Lab2;
```

```
public class FinalExam(string title, int ticketCount, List<string>
   commission) : Exam(title, ticketCount), IProtocolable
{
    public List<string> Commission { get; } = commission;
    public override void Conduct(Student student)
    {
        base.Conduct(student);
        if (Commission.Count == 0) throw new InvalidOperationException(
   "Final exam has no commission.");
    }
    public string MakeProtocol() => $"Protocol: {Title}, commission: {
   string.Join(", ", Commission)}";
}
```

```
namespace Lab2;
public class Result(Student student, Assessment assessment, int score)
{
    public Student Student { get; } = student;
    public Assessment Assessment { get; } = assessment;
    public int Score { get; } = score;
    public override string ToString() => $"Result: {Student} – {
   Assessment}: {Score}";
}
```

```
namespace Lab2;
public interface IConductable
{
    void Conduct(Student student);
}
```

```
namespace Lab2;
public interface IAssessable
{
    Result Assess(Student student, int score);
}
```

```
namespace Lab2;
public interface IProtocolable
{
    string MakeProtocol();
}
```

```
namespace Lab2;
public interface IResultArchive
{
    void Save(Result result);
    List<Result> FindByStudent(Student student);
}
```

```
namespace Lab2;
public class InMemoryResultArchive : IResultArchive
{
    private List<Result> results = new();
    public void Save(Result result) => results.Add(result);
    public List<Result> FindByStudent(Student student) => results.Where
   (r => r.Student == student).ToList();
}
```

```
namespace Lab2;
public class Session(IResultArchive archive)
{
    private  List<Assessment> assessments = new();
    public void Add(Assessment assessment) => assessments.Add(
   assessment);
    public void ConductAll(Student student, List<int> scores)
    {
        if (scores.Count != assessments.Count) throw new
   ArgumentException("Scores count must match assessments count.");
        for (int i = 0; i < assessments.Count; i++)
        {
            assessments[i].Conduct(student);
            archive.Save(assessments[i].Assess(student, scores[i]));
        }
    }
}
```

```
using Lab2;


// init
var stud = new Student("ivan");
```

```
var test = new Test("linal", questionCount: 10);
var exam = new  Exam("matanal", ticketCount: 5);
var finalExam = new FinalExam("Diplom",ticketCount:20 ,commission: ["
   teacher_good", "teacher_bad"]);

var archive = new InMemoryResultArchive();
var session = new Session(archive);

// Пример LSP --  все три как Испытание.
session.Add(test);
session.Add(exam);
session.Add(finalExam);



// Провести все экзамены, у всех разные реализации этого.
session.ConductAll(stud, [5,4,3]);

foreach (Result r in archive.FindByStudent(stud)){
    Console.WriteLine(r.ToString());
}

var stud2 = new Student("Sander");
session.ConductAll(stud2, [2,2,2]);



foreach (Result r in archive.FindByStudent(stud2)){
    Console.WriteLine(r.ToString());
}
```

# Результат запуска (тестирование функциональности)

```
Result: ivan – linal: 5
Result: ivan – matanal: 4
Result: ivan – Diplom: 3
Result: Sander – linal: 2
Result: Sander – matanal: 2
Result: Sander – Diplom: 2
```